

Employee Management System Database Schema and Database Code Submission

1. Introduction and Application Relevance

The selected application area for this project is an Employee Management System (EMS) designed for HR teams and organizational managers. Modern organizations require a structured and reliable system to manage employee profiles, department assignments, attendance records, leave management, and payroll processing.

A relational database system is ideal for this application because it ensures:

Data integrity

Strong relationships between entities

Accurate reporting capabilities

Reduced redundancy through normalization

The database developed in this project supports the following functionalities:

Employee profile and record management

Department and manager assignment

Daily attendance tracking

Leave request lifecycle management (Pending, Approved, Rejected)

Monthly payroll processing and analytics

2. Database Schema Design (Normalized)

The database schema has been designed according to Third Normal Form (3NF) to ensure minimal redundancy, consistency, and maintainability.

The schema consists of the following tables:

app_users

departments

employees

attendance_logs

leave_requests

payroll_records

Each table has clearly defined primary keys, foreign keys, and constraints to maintain data integrity.

3. Table Descriptions 3.1 app_users

This table stores login credentials and role information for system users.

Primary Key

user_id

Constraints

Unique email address

Valid role values (Admin, HR, Manager, Employee)

Purpose Separates authentication data from employee profile information.

3.2 departments

This table stores department-level information within the organization.

Primary Key

department_id

Foreign Key

manager_user_id app_users.user_id

Constraints

Unique department name

Purpose Maintains department records and identifies department managers.

3.3 employees

This table stores employee HR profile information.

Primary Key

employee_id

Foreign Keys

user_id app_users.user_id

department_id departments.department_id

Constraints

Unique user_id

Unique phone number

Salary must be greater than zero

Valid employment status values

Purpose

Links employee identity with login accounts and departments.

3.4 attendance_logs

This table stores daily attendance data for employees.

Primary Key

attendance_id

Foreign Key

employee_id employees.employee_id

Constraints

One attendance record per employee per day

Valid attendance status

check_out must be greater than or equal to check_in

Purpose

Tracks employee presence and working hours.

3.5 leave_requests

This table stores leave applications submitted by employees.

Primary Key

leave_id

Foreign Key

employee_id employees.employee_id

Constraints

Valid leave type

Valid leave status

end_date must be greater than or equal to start_date

Derived Column

days_requested

Purpose

Manages employee leave requests and approval workflows.

3.6 payroll_records

This table stores payroll information for employees.

Primary Key

payroll_id

Foreign Key

employee_id employees.employee_id

Constraints

One payroll record per employee per month

Salary components must be non-negative

Derived Column

net_salary

Purpose

Stores salary breakdown and payment details for employees.

4. Normalization Notes

The database follows Third Normal Form (3NF) principles.

Key normalization decisions include:

User authentication data is stored separately from employee profile data.

Department information is stored once and referenced through foreign keys.

Transactional data such as attendance, leaves, and payroll are stored in dedicated tables.

Repeating groups are avoided.

Relationships between tables are maintained using foreign keys.

This approach ensures data consistency, reduced redundancy, and easier maintenance.

5. Entity Relationship (ER) Diagram

The ER diagram for the system is provided using Mermaid syntax.

File included in submission:

er-diagram.mmd

The diagram illustrates relationships between:

Users

Departments

Employees

Attendance logs

Leave requests

Payroll records

To render the diagram:

Open Mermaid Live Editor

Upload the .mmd file

Export as PNG or SVG for documentation

6. Database Implementation

The database is implemented using PostgreSQL.

The SQL implementation file included is:

employee_management_db.sql

This file contains the following sections:

1. Database Setup

Drop existing tables (for clean reruns)

2. Table Creation

CREATE TABLE statements

Primary key definitions

Foreign key relationships

Unique constraints

Check constraints

3. Sample Data Population

Each table contains at least 5 sample records for testing and demonstration.

7. Data Manipulation and Querying

The SQL script includes multiple queries demonstrating database functionality.

7.1 Data Manipulation Queries

These queries demonstrate insert, update, and delete operations.

DML 1 — Insert Data

Insert a new user and linked employee record using a Common Table Expression (CTE).

DML 2 — Update Data

Update employee salary information.

DML 3 — Delete Data

Delete an attendance record.

7.2 Data Retrieval Queries

At least five retrieval queries are included.

Q1 — Employee Directory

Displays employee information with department and role.

Features used:

JOIN

ORDER BY

Q2 — Department Headcount

Displays total employees per department.

Features used:

GROUP BY

COUNT

Q3 — Monthly Payroll Totals

Calculates total payroll expenses per month.

Features used:

GROUP BY

Aggregate functions

Q4 — Attendance Summary

Displays attendance statistics per employee.

Features used:

JOIN

Conditional aggregation

Q5 — Leave Requests Report

Displays leave applications with department information.

Features used:

JOIN

FILTER

ORDER BY

7.3 Complex Queries

Two advanced queries are included.

CQ1 — Highest Paid Employee per Department

Uses:

Common Table Expression (CTE)

Window Function (ROW_NUMBER)

Purpose:

Identify the highest-paid employee in each department.

CQ2 — Employees Above Department Average Salary

Uses:

Correlated subquery

Purpose:

Identify employees whose salary is above the average salary of their department.

8. How to Run the Database

Follow these steps to run the project.

Create a PostgreSQL database.

Example:

```
CREATE DATABASE ems_db;
```

Open a SQL client such as:

pgAdmin

DBeaver

psql

Execute the SQL file:

employee_management_db.sql

Run the query sections to view the results.

9. Evidence / Screenshots Required

Include the following screenshots in the final report.

Successful table creation

Data preview for each table

DML query execution results

Retrieval query outputs

Complex query outputs

ER diagram visualization

Recommended screenshot naming format:

Screenshot_01_CreateTables.png Screenshot_02_UsersData.png

Screenshot_03_EmployeesData.png Screenshot_04_RetrievalQueries.png

Screenshot_05_ComplexQueries.png Screenshot_06_ERDiagram.png 10. Mapping to Evaluation Criteria Correctness and Efficiency

Fully normalized schema

Strong relationships between tables

Proper constraints and keys

Implementation Quality

Clean, executable PostgreSQL SQL script

Structured DDL, DML, and query sections

Complexity and Creativity

Use of advanced SQL features

Window functions and subqueries

Documentation and Presentation

Structured documentation

Clearly labeled screenshots

ER diagram included

Best Practices

Consistent naming conventions

Proper data types

Readable SQL formatting

11. Files Included in Submission Folder

The submission ZIP file contains the following:

employee_management_db.sql

er-diagram.mmd

Database_Documentation.md